OXFORD

The Computer Journal, 2025, **68**, 998–1012
https://doi.org/10.1093/comjnl/bxaf019
Advance access publication date 2 April 2025
Original Article

F2PQNN: a fast and secure two-party inference on quantized convolutional neural networks

Jinguo Li [1], Peichun Yuan [1], Jin Zhang [2,*], Sheng Shen [3], Yin He [1], Ruyang Xiao [1]

[1]College of Computer Science and Technology, Shanghai University of Electric Power, Shanghai 201306, China
[2]School of Computer and Communication Engineering, Changsha University of Science and Technology, Changsha 410114, China
[3]College of Engineering, University of California, Santa Barbara, CA 93106, United States

*Corresponding author. E-mail: mail_zhangjin@163.com

Abstract

The machine learning as a service (MLaaS) paradigm has been widely adopted across various applications. However, it also raises significant privacy concerns, particularly regarding the exposure of input data and trained models. Two-party computation in convolutional neural network (CNN) inference has emerged as a promising solution to address these privacy issues in MLaaS. Nevertheless, most existing privacy-preserving CNN architectures rely on computationally expensive encryption methods, resulting in prolonged inference times and increased communication overhead. In this paper, we propose F2PQNN, a fast and secure two-party inference framework for quantized CNNs. To minimize reliance on computationally intensive encryption, F2PQNN utilizes two non-colluding servers and integrates secret sharing with oblivious transfer techniques. Furthermore, F2PQNN incorporates quantization techniques, along with batching and asynchronous computation, to significantly accelerate inference predictions. We evaluate the performance of F2PQNN on the MNIST, Fashion-MNIST, CIFAR-10, and STL-10 datasets. Experimental results demonstrate that F2PQNN outperforms existing solutions, achieving a 9.14× speedup and reducing communication overhead by 59.8× on the MNIST dataset.

1. INTRODUCTION

Machine learning as a service (MLaaS) has gained widespread adoption due to its numerous advantages, with major companies such as AWS SageMaker, Google AI, and Azure Machine Learning offering comprehensive neural network prediction services to clients. In a traditional MLaaS setup, the model owner provides a well-trained neural network model, and users access inference services through APIs. However, this approach raises significant privacy concerns, as cloud servers may gain access to users' private inputs and raw inference results [1, 2]. Additionally, large amounts of private data are often used for model training on the cloud, making it crucial to ensure the confidentiality of both the data and trained models [3].

To address these issues, researchers have primarily focused on three key techniques: homomorphic encryption (HE) [4–6], differential privacy (DP) [7, 8], and secure multiparty computation (MPC) [9–11]. However, each of these approaches has limitations: (i) HE often suffers from computational inefficiency and struggles to support complex nonlinear operations (e.g. CryptoNets [12]). (ii) DP typically requires the introduction of significant noise, which can compromise data accuracy and utility. Furthermore, DP is vulnerable to statistical attacks and re-identification attempts, especially for small-scale datasets (e.g. PATE [7], Bu19 [8]). In contrast, secure MPC enables multiple participants to perform computations without revealing their private inputs, allowing them to jointly train models while preserving data privacy. MPC offers greater flexibility, higher computational efficiency, and improved accuracy, making it a promising solution. Therefore, this paper focuses on secure MPC technologies.

Several MPC-based techniques have been applied to convolutional neural networks (CNNs) [13–15]. These studies primarily adopt a hybrid approach, combining additive secret sharing (AdSS) [16] for linear layers (e.g. convolutional and fully connected layers) with garbled circuits (GC) [14, 17, 18] for non-linear layers (e.g. ReLU layers). While AdSS supports fast addition and multiplication, enabling near-plaintext speed computation for linear layers, GC incurs high computation and communication costs, making secure computation for non-linear layers prohibitively expensive [14]. As a result, most related works focus on reducing the overhead of non-linear operations to achieve high-performance secure inference.

Previous algorithms have attempted to optimize the performance of non-linear operations by minimizing the cost of rectified linear unit (ReLU) operations. Strategies include reducing the number of ReLUs (e.g. DeepReduce [19], CryptoNAS [20]), replacing ReLUs with polynomial functions (e.g. CryptoNets [12], Delphi [14], FPPNe [21]), using binary weights and activations (e.g. binary neural networks [22]), or designing secure comparison protocols for ReLUs with purely arithmetic operations (e.g. PCNNCEC [11]). However, these techniques often sacrifice model comprehensibility and face scalability challenges, leading to accuracy losses in large networks and datasets such as ImageNet.

Although quantization techniques offer potential solutions for secure and efficient neural network prediction [23–25], they often suffer from low quality in MLaaS due to introduced errors. For instance, inappropriate quantization scales can destabilize the entire secure inference process.

To address these challenges, we propose F2PQNN, a fast and secure two-party inference method for quantized CNNs. On one

Received: June 23, 2024. **Revised:** January 17, 2025. **Accepted:** January 26, 2025

hand, F2PQNN enables secure inference services in the cloud, protecting sensitive user data and trained model parameters from unauthorized access. On the other hand, by leveraging two-party comparison and quantization techniques, F2PQNN significantly reduces communication overhead in the two-party computation (2PC) setup and improves computational efficiency through fast secret exchanges. Our contributions are summarized as follows:

- We propose F2PQNN, a cloud-based outsourced secure service that integrates cryptographic techniques with advancements in machine learning. F2PQNN implements a versatile cloud service framework that supports various CNN models and provides high-performance secure inference services.
- To accelerate the generation of offline triplets [26], we develop the single instruction multiple data (SIMD) technique [27] with asynchronous computation. Additionally, we design a 2PC-based approach for the ReLU function, combining arithmetic-to-binary shared conversion with oblivious transfers, eliminating the need for GC.
- We conduct experiments on datasets of varying scales to demonstrate the effectiveness and efficiency of F2PQNN. Different network architectures are adopted to verify its exceptional scalability. Furthermore, we identify the optimal data width by investigating the accuracy loss caused by quantization schemes.

The remainder of this paper is organized as follows: Section 2 reviews prior related works. Section 3 defines the symbols and primitives associated with encryption tools, neural networks, and quantization. Section 4 presents the architecture and model overview of F2PQNN. Secure protocols for linear and nonlinear computations are detailed in Section 5. In Section 6, we analyze the accuracy loss introduced by the quantization scheme. The security of F2PQNN is examined in Section 7. Finally, Section 8 implements our protocol and evaluates its performance, followed by a summary of the paper in Section 9.

2. RELATED WORK

Several privacy-preserving machine learning approaches have primarily focused on traditional algorithms such as decision trees, naive Bayes classification, and logistic regression [28–30]. However, with the rapid development of CNNs, various security protocols based on hybrid encryption techniques—such as HE, GC, and secret sharing—have been explored. Most of these protocols mitigate data security risks by outsourcing encrypted data to cloud servers for training or inference tasks. In the following sections, we summarize the advancements in these techniques from two perspectives: encryption protocol-based optimization methods and machine learning based optimization methods.

2.1. Encryption protocol-based optimization methods

For HE and DP-based methods, Bachrach et al. [12] applied leveled homomorphic encryption (LHE) to CryptoNets. However, LHE imposes limitations on the number of addition and multiplication operations, making it unsuitable for deep neural networks with multiple nonlinear layers. PATE [7] employed DP techniques in generative adversarial networks, but its performance degrades when noise is added to diverse data, leading to reduced accuracy. In summary, HE and DP schemes still face significant challenges in handling nonlinear operations on ciphertext.

To address these challenges, researchers have increasingly focused on multi-party protocols or combinations of similar protocols for inference tasks. For instance, SecureML [31] uses AdSS and offline-generated multiplication triples (MTs) for multiplication operations, while approximating nonlinear activation functions with piecewise linear functions via GC. Despite these optimizations, communication efficiency remains a bottleneck [32]. MiniONN [17] improved matrix multiplication protocols by employing SIMD batch processing techniques for LHE operations, enabling linear transformations in the offline phase. Building on this, GAZELLE [13] enhanced the efficiency of CNN convolutional computations using encapsulated HE, achieving a 20× speedup over MiniONN. Delphi [14] further improved GAZELLE's performance by replacing ReLU with square functions.

Unlike polynomial approximation methods, FPPNet [21] converts multiplicative sharing into additive sharing, which can lead to rapidly escalating computational errors as input values increase. To address this, an oblivious transfer (OT)-based secure protocol [15] was proposed for nonlinear computations, demonstrating significantly better performance compared to Delphi. Sonic [9] employs lightweight cryptographic primitives for secure inference, but its linear layer multiplication operations require multiple rounds of communication. Pio [33] improved linear operations in the GAZELLE protocol to achieve efficient neural network outsourcing inference, though ciphertext computations still incur significant overhead. PCNNCEC [11] designed a secure comparison protocol using pure arithmetic operations, but its inputs can be inferred from available symbols. Cenia [10] developed a low-interaction secure comparison protocol with arithmetic secret sharing, applicable for implementing ReLU and Max Pooling, though it still suffers from high communication overhead.

GC [34] have also been applied to CNN inference tasks. These GC-based methods generate correlated randomness using MTs and OT techniques. For instance, SecureNN [35] and Cryptflow [36] developed protocols for nonlinear functions involving third parties, enabling secure inference with three or even four servers. However, compared to dual-server architectures, these approaches often introduce additional communication costs and deployment complexities in practical scenarios.

2.2. Machine learning-based optimization methods

In quantized neural networks (QNNs), training parameters are typically encoded as fixed-point numbers, represented by low-bit-width floating-point values (e.g. 8 bits). The computational costs of encryption tools in QNNs generally increase linearly with the input bit width. For example, XONN [18] constrained weights and activation values to the binary set $\{-1, 1\}$, replacing multiplication operations with XNOR operations implemented via GC. This quantization approach reduces the time consumption of encrypted data computations. QUOTIENT [22] extended this idea to ternary $\{-1, 0, 1\}$ neural networks by transforming ternary multiplication into two binary multiplications and designing a 1-out-of-2 OT protocol. DeepSecure [32] introduced a data projection preprocessing technique to generate low-dimensional data, reducing model size.

To further reduce the computation and communication costs of GC, some techniques exploit the sparse characteristics of deep learning models. For instance, SecureQ8 [23] constructed three-party random secret sharing for INT8 quantization under different threat models. ABNN2 [24] presented a secure two-party

framework for QNN inference with arbitrary bit widths. However, these solutions struggle to efficiently manage the substantial communication overhead between the two parties, a challenge that becomes more pronounced as neural networks scale. Although AQ2PNN [25] implemented secure 2PC on FPGA, it still suffers from high latency.

3. PRELIMINARIES

3.1. Additive secret sharing

Secret sharing is a cryptographic technique used to distribute a confidential value among multiple parties, ensuring that no individual share reveals any information about the secret. The secret can only be reconstructed when a sufficient number of shares are combined. In AdSS, an ℓ-bit secret value x is split between two parties by dividing it into two shares, $\langle x\rangle_0$ and $\langle x\rangle_1$, within the ring $\mathbb{Z}_{2^\ell}$. These shares satisfy the equation $\langle x\rangle_0 + \langle x\rangle_1 = x \,(\mathrm{mod} 2^\ell)$, where $\langle x\rangle_0$ and $\langle x\rangle_1$ are randomly chosen. One party receives $\langle x\rangle_0$, and the other receives $\langle x\rangle_1$. Neither party can independently determine the value of x.

When two secret shares x and y are present, secure computation techniques enable the two parties, P_0 and P_1, to perform operations on their shares. For example, addition/subtraction: $\langle z\rangle_i = \langle x\rangle_i \pm \langle y\rangle_i$, where $i \in \{0,1\}$; multiplication by a public constant: $\langle z\rangle_i = \eta \cdot \langle x\rangle_i$, where $i \in \{0,1\}$; multiplication of shares: $\langle x \cdot y\rangle_i$. To reduce online communication overhead, Beaver's triples [26] are commonly used. These triples consist of three correlated secrets $\langle a\rangle_i$, $\langle b\rangle_i$, and $\langle c\rangle_i$, where $\langle c\rangle_i = \langle a \cdot b\rangle_i$, $i \in \{0,1\}$. The participants first compute $\langle d\rangle_i = \langle x - a\rangle_i$ and $\langle e\rangle_i = \langle y - b\rangle_i$, then reconstruct d and e. Finally, using Equation (1), the shares $\langle x \cdot y\rangle_i$ can be obtained locally as follows:

$$\langle x \cdot y\rangle_i = d \cdot e + d \cdot \langle b\rangle_i + e \cdot \langle a\rangle_i + \langle c\rangle_i, i \in \{0,1\} \tag{1}$$

3.2. Quantization neural networks

Quantization is a technique used to improve the efficiency of neural networks by reducing the bit width of parameters, such as converting 32-bit floating-point numbers to 8-bit integers [37]. The quantized real number x can be represented as $Q^f(x) := \lfloor x \cdot 2^f \rceil$, where f is a positive integer specifying the precision. This approach reduces both model size and bandwidth requirements by a factor of 4 (as shown in Fig. 1). Additionally, integer operations are computationally less expensive than floating-point operations on most processor cores, further accelerating network inference.

Various quantization methods have been proposed [38–42]. However, most of them [39] introduce significant redundancy to achieve high accuracy. To obtain better performance, some approaches [40–42] focus exclusively on quantizing weights but neglect computational efficiency. Although techniques such as binary and ternary weight networks or shift networks [37, 43, 44] constrain weights to 0 or powers of 2, replacing multiplications with shifts. However, shift operations do not outperform multiplication-addition instructions, leaving the cost of multiplication high.

A key requirement for quantization methods is the ability to perform all arithmetic operations using only integer calculations on quantized values. In our protocol, linear operations are particularly advantageous for constructing secret-sharing-based MPC protocols due to their compatibility with quantization [45]. During testing, integer calculations are used exclusively, while floating-point calculations are employed during training. Both weights and input data are quantized before convolution. For layers using batch normalization (BN), BN is "folded" into the weights prior to quantization. Finally, the mean shift of activation values is corrected [46], enhancing the performance of the quantized model.

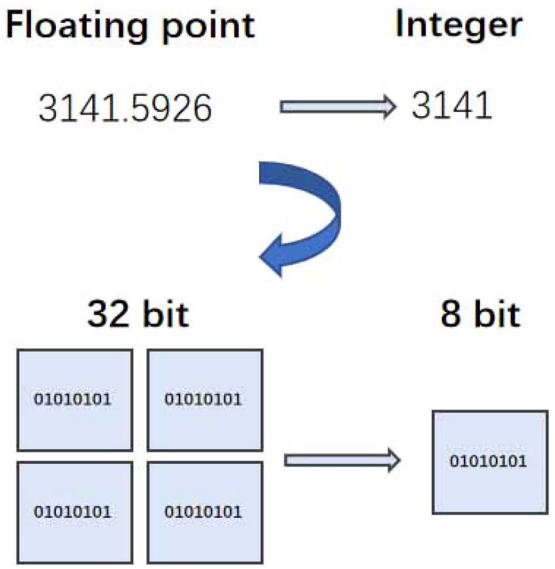


Figure 1. Conversion of a 32-bit floating-point quantity to an 8-bit integer.

Table 1 provides a comprehensive summary of the notations used in our scheme.

4. SYSTEM SETTINGS

This section outlines the system architecture and threat model of our secure inference framework.

4.1. System model

Our secure inference system enables the deployment of a trained QNN model on the cloud. As illustrated in Fig. 2, the system incorporates SCONV, SMP, SReLU, and SFC as security layers. The symbols X, z, and W denote input data, intermediate results, and weight parameters, respectively. The subscripts 0 or 1 indicate the association of secret-shared information with S_0 or S_1. The system involves three distinct roles:

1. Servers: Our system includes two non-colluding servers S_0 and S_1. These servers can belong to different organizations, and any collusion between them would damage their reputation. Similar to prior works, it is assumed that the servers are aware of the layer types, sizes, and numbers. A series of secure protocols are executed by the servers to perform secure inference tasks.
2. User: The user holds queries X, and receives secure inference services. X is transformed into 8-bit integers, and secret shares $\langle X\rangle_0$ and $\langle X\rangle_1$ are generated. These shares are then provided to S_0 and S_1. The cloud servers collaboratively execute secure inference system and produce secret shares $\langle z\rangle_0$ and $\langle z\rangle_1$ back to the user. At last, the user reconstructs the inference result z.
3. Model Owner: The model owner possesses a trained QNN model W, which includes quantized weights and parameters for various layers. To protect the privacy of QNN, the owner

Table 1. Notations

Notation	Description
S_P	The cloud server, $p \in \{0, 1\}$
X, Z, W	Input data, intermediate results, and weight parameters
$\langle x \rangle_0$, $\langle x \rangle_1$	Two parties to x secret shares
$[x]_0$, $[x]_1$	The distribution of the secret between two parties following the encryption of x using the key pair (pk, sk)
$\boxplus$, $\boxtimes$	$[x] \boxplus [y] = [x + y]$; $[x] \boxtimes [y] = [x \times y]$
a, b, c, d, e	A Beaver triple with $c = a \times b$, $d = x - a$, $e = y - b$
$Q^f(x)$	The quantification representation of x, f is the quantization precision

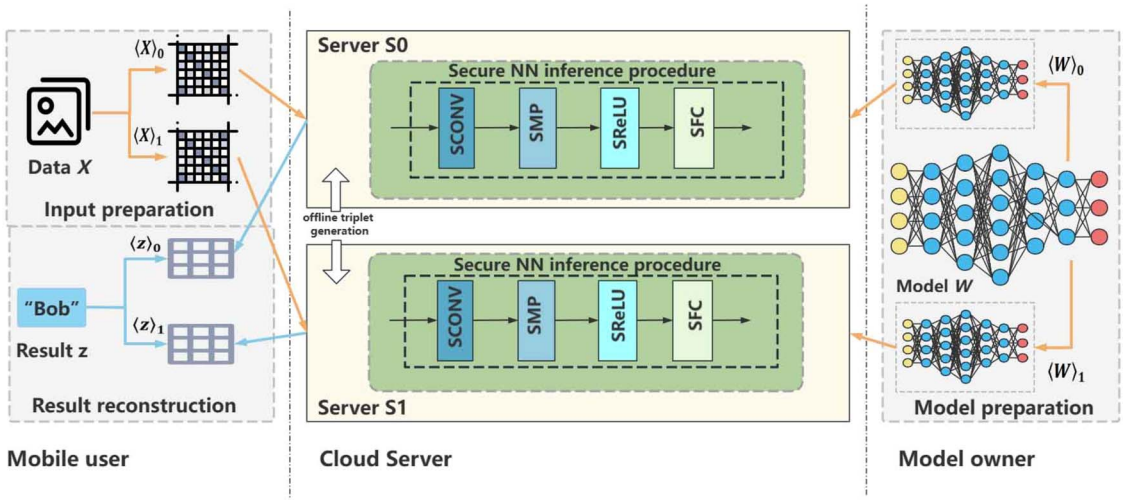


Figure 2. A quantized CNN prediction system based on two non-colluding cloud servers.

generates secret shares $\langle W \rangle_0$ and $\langle W \rangle_1$ before deploying them to the cloud servers S_0 and S_1.

4.2. Threat model

Similar to [13, 14, 17, 31], our threat model assumes that both servers are semi-honest and non-colluding. We consider the following types of attackers:

- Internal attackers: Internal attackers include the cloud server and clients, which are considered semi-honest entities. These entities always faithfully adhere to the protocol specifications while attempting to infer each other's private information from the records of the protocol.
- External attackers: External attackers can eavesdrop on the communication channel between the cloud server and client, and attempt to infer the private information from the transmitted data.

In Section 6, we have demonstrated that the proposed framework maintains security under semi-honest corruption. Since most secure neural network inference methods involve recommunication after evaluating each layer of the neural network, and existing neural network architectures are public known, our framework aims to protect the sensitive data of clients and the model parameters provided by the model owner. similar to previous secure inference works [13, 14, 17, 31], black-box attacks on neural networks are not considered in this paper.

5. DETAILS FOR F2PQNN

In this section, we introduce the proposed secure protocol F2PQNN. For each stage of online prediction, our protocol operates through secret sharing between two servers. Each layer of the neural network has its corresponding secure protocol, and the secure layers are organized in a pipeline fashion. Each unit is designed to accommodate secret sharing techniques.

5.1. Preparation

The convolutional and FC layers involve numerous inner product computations. To reduce the online computation cost of secure sharing, S_0 and S_1 first prepare multiplication triplets [26]. As illustrated in Algorithm 1, S_0 holds $\langle a \rangle_0$ and $\langle b \rangle_0$, while S_1 holds $\langle a \rangle_1$ and $\langle b \rangle_1$. The values satisfy $\langle a \rangle_0 + \langle a \rangle_1 = a$ and $\langle b \rangle_0 + \langle b \rangle_1 = b$. The objective is to calculate the shared form of the multiplication between a and b. Specifically, S_0 obtains $\langle c \rangle_0$ and S_1 obtains $\langle c \rangle_1$, where $\langle c \rangle_0 + \langle c \rangle_1 = c = a \cdot b = (\langle a \rangle_0 + \langle a \rangle_1)(\langle b \rangle_0 + \langle b \rangle_1)$. The following steps outline the implementation process:

Step 1: S_0 encrypts $\langle a \rangle_0$ and $\langle b \rangle_0$, then returns them to S_1.

Step 2: S_1 calculates the products of $\langle a \rangle_0$ with $\langle b \rangle_1$, and $\langle a \rangle_1$ with $\langle b \rangle_0$ by multiplicative homomorphism encryption. Then it computes the sum of these products by additive homomorphism encryption, and adds a random number r into the result V_3.

Step 3: S_1 locally computes $\langle c \rangle_1 = \langle a \rangle_1 \cdot \langle b \rangle_1 - r$, and then sends V_3 to S_0.

Step 4: S_0 decrypts V_3 to obtain the plaintext v_3. Thus, $\langle c \rangle_0 = v_3 + \langle a \rangle_0 \cdot \langle b \rangle_0 = a \cdot b - \langle a \rangle_1 \cdot \langle b \rangle_1 + r$.

Step 5: The servers reconstruct the result of shared multiplication $c = \langle c \rangle_0 + \langle c \rangle_1$. (Neither S_0 nor S_1 can learn c.)

5.1.1. *Optimized execution of the multiplicative triplets*

The use of SIMD data packaging and asynchronous computation can improve the speed of MT generation. SIMD is a parallel

Algorithm 1 Multiplication triple generation

Input: S_0 and S_1 hold the shares of a and b, where $a, b \in \mathbb{Z}_{2^\ell}$. S_0 holds the key pair (pk, sk), and S_1 holds the public key pk.
Output: S_0 and S_1 obtain the shares of c, where $c = a \cdot b \bmod 2^\ell$.

1: S_0 encrypts the shares of a and b to obtain $[a]_0$ and $[b]_0$;
2: S_0 sends $[a]_0$ and $[b]_0$ to S_1;
3: S_1 computes $V_1 = [a]_0 \boxtimes [b]_1$, $V_2 = [a]_1 \boxtimes [b]_0$;
4: S_1 sends $V_3 = V_1 \boxplus V_2 \boxplus [r]$ to S_0, where $r \in \mathbb{Z}_{2^\ell}$;
5: S_1 sets $\langle c\rangle_1 = (\langle a\rangle_1 \cdot \langle b\rangle_1 - r) \bmod 2^\ell$;
6: S_0 decrypts V_3 to get $v_3 = \langle a\rangle_0 \cdot \langle b\rangle_1 + \langle a\rangle_1 \cdot \langle b\rangle_0 + r$;
7: S_0 sets $\langle c\rangle_0 = v_3 + \langle a\rangle_0 \cdot \langle b\rangle_0 = (a \cdot b - \langle a\rangle_1 \cdot \langle b\rangle_1 + r) \bmod 2^\ell$;
8: Return ($\langle a\rangle_0$,$\langle b\rangle_0$,$\langle c\rangle_0$) and ($\langle a\rangle_1$,$\langle b\rangle_1$,$\langle c\rangle_1$).

computing technique that allows concurrent execution of the same operation on multiple data elements simultaneously. When generating multiplicative triples, the system leverages the SIMD instruction set to enable the execution of the same computational operations on multiple data elements concurrently. Thereby it can enhance computation efficiency.

We package n shares ($\langle a\rangle_{0,1}$, $\langle b\rangle_{0,1}$, $\langle a\rangle_{1,1}$, $\langle b\rangle_{1,1}$), ($\langle a\rangle_{0,2}$, $\langle b\rangle_{0,2}$, $\langle a\rangle_{1,2}$, $\langle b\rangle_{1,2}$)...($\langle a\rangle_{0,n}$, $\langle b\rangle_{0,n}$, $\langle a\rangle_{1,n}$, $\langle b\rangle_{1,n}$) as ($\langle A\rangle_0$, $\langle B\rangle_0$, $\langle A\rangle_1$, $\langle B\rangle_1$). The packaged multiplicative triples ($\langle A\rangle_0$, $\langle B\rangle_0$, $\langle C\rangle_0$), ($\langle A\rangle_1$, $\langle B\rangle_1$, $\langle C\rangle_1$) can be obtained through homomorphic operations on the packaged data using SIMD. Finally, by unpacking the result, we can obtain n multiplication triplets.

Furthermore, in the offline phase, most schemes use parallel computation, i.e. S_0 and S_1 must wait for an intermediate result from the other party when generating a triplet. In contrast, asynchronous computing allows multiple tasks to be performed simultaneously without waiting for the previous task to complete. To speed up the calculation, we adopt an asynchronous computation scheme. For instance, in Step 1, S_0 encrypts $\langle a\rangle_0$ and sends it to S_1. S_0 does not need to wait for $\langle b\rangle_0$ to be encrypted before sending the ciphertext. In Step 3, the local computation of S_1 can be performed simultaneously while sending V_3, and S_1 does not need to wait for the calculation of $\langle c\rangle_1$ before sending V_3. Asynchronous computing reduces latency by minimizing wait times compared to synchronous computing.

5.2. Secure linear layers computation

The secure linear layer computation can be divided into convolution computation and FC-layer computation. It focuses on matrix multiplication operations, thus we explore protocols for secure implementing matrix multiplication. As shown in Algorithm 2, we use a 1×4 input vector X and a 4×4 weight matrix W as examples in Fig. 3 to illustrate the safe matrix multiplication process. Take a set of secret shares of X and W as input respectively, i.e., S_0 gets the inputs $\langle X\rangle_0$ and $\langle W\rangle_0$, and S_1 gets the inputs $\langle X\rangle_1$ and $\langle W\rangle_1$. The vector X serves as either the activation output of the preceding ReLU function or represents the user's original input. The vector uses the calculation in 3.1 to obtain d and e. The output result is obtained through Equation (1), i.e. S_0 obtains the output $\langle z\rangle_0$, and S_1 obtains the output $\langle z\rangle_1$. We can observe that the secure matrix multiplication technique can resist revealing any original values during the computation. However, the introduction of quantization in our scheme may accumulate a certain error when using quantized numbers to calculate matrix multiplication. Section 6 below provides a detailed analysis of the solution to this potential problem.

Algorithm 2 Secure matrix multiplication

Input: S_0 and S_1 hold the shares of W, X and triplet (a, b, c), which are generated by Algorithm 1.
Output: S_0 and S_1 obtain the shares of z, where $z = X \cdot W$.

1: S_i locally computes $\langle d\rangle_i = \langle x - a\rangle_i$ and $\langle e\rangle_i = \langle y - b\rangle_i$, then reconstructs d and e. Thus, S_i obtains d and e, where $i \in \{0, 1\}$.
2: S_0 sets $\langle z\rangle_0 = \langle X\rangle_0 \cdot \langle W\rangle_0 = d \cdot \langle b\rangle_0 + e \cdot \langle a\rangle_0 + \langle c\rangle_0$.
3: S_1 sets $\langle z\rangle_1 = \langle X\rangle_1 \cdot \langle W\rangle_1 = d \cdot \langle b\rangle_1 + e \cdot \langle a\rangle_1 + \langle c\rangle_1 + d \cdot e$.

5.3. Secure truncation computation

When both parties perform secure matrix multiplication using fixed-point values, it is necessary to handle results in double-precision format. The number of digits in the output increases due to matrix multiplication. For example, let the shared values are all expressed as ℓ-bit fixed-point values with d-bit precision, the multiplied result of two fixed-point numbers can be $2d$-bit precision. Hence, truncation is required to ensure consistency in the number of digits between the input and output values. Since the traditional truncation method can lead to significant overhead and large errors due to overflow in the sum of secret shares, we develop the truncation method outlined in CHEETAH [47]. As illustrated in Algorithm 3, given an unsigned ℓ-bit integer x, with $\langle x\rangle_0$ and $\langle x\rangle_1$ representing the secret-shared shares of x, where $\langle x\rangle_0 + \langle x\rangle_1 = x \left(\mathbf{mod} 2^\ell\right)$, we define $w = \mathbf{1}\{x_0 + x_1 > 2^\ell - 1\}$. Here, c is a 1-bit integer (0 or 1). We can express the result as $(x_0 \gg f) + (x_1 \gg f) - w \cdot 2^\ell = (x \gg f) - c$, where $(x \gg f)$ means that the binary bits of x are shifted to the right by f bits.

Algorithm 3 Secure truncation

Input: S_0 and S_1 hold the shares of z, where $z \in \mathbb{Z}_{2^\ell}$. Truncation bit f, where $f \in \mathbb{Z}_{2^\ell}$.
Output: S_0 and S_1 obtain the shares of z′, where $z' = z \gg f$ or $z' = (z \gg f) - 1$.

1: S_0 sets $\langle z'\rangle_0 = (\langle z\rangle_0 \gg f) - \langle w\rangle_0 \cdot 2^{\ell - f}$;
2: S_1 sets $\langle z'\rangle_1 = (\langle z\rangle_1 \gg f) - \langle w\rangle_1 \cdot 2^{\ell - f}$.

Maintaining consistency between the output and input bit widths is crucial for the accurate implementation of quantized CNN models. From a quantization perspective, truncation can be minimized by postponing it until after the summation. Since truncation is the most computationally expensive part of quantization safe matrix multiplication, this approach not only reduces truncation errors but also improves efficiency.

5.4. Secure nonlinear layers computation

5.4.1. *Secure comparison computation*

Data comparison is a pivotal and challenging step, particularly when the values being compared are encrypted and exchanged between both parties. In current research, this operation is commonly implemented using Yao's GC [48]. To reduce communication overhead, we propose an efficient approach for secure comparison based on secret sharing and OT techniques. It supports secure 2PC ReLU and 2PC MaxPool operations, and can enhance the versatility and applicability. Our secure comparison protocol is outlined as follows:

At first, arithmetic shares are converted to binary shares (A2B). They are then divided into U groups using the $\|$ notation. For instance, consider the case of an 8-bit integer arithmetic share (INT8), which can be classified into five groups: $x \leftarrow x_7 \| x_6 \| x_5x_4 \| x_3x_2 \| x_1x_0$, where $x \in \mathbb{Z}_{2^8}$. The correlation of the two most

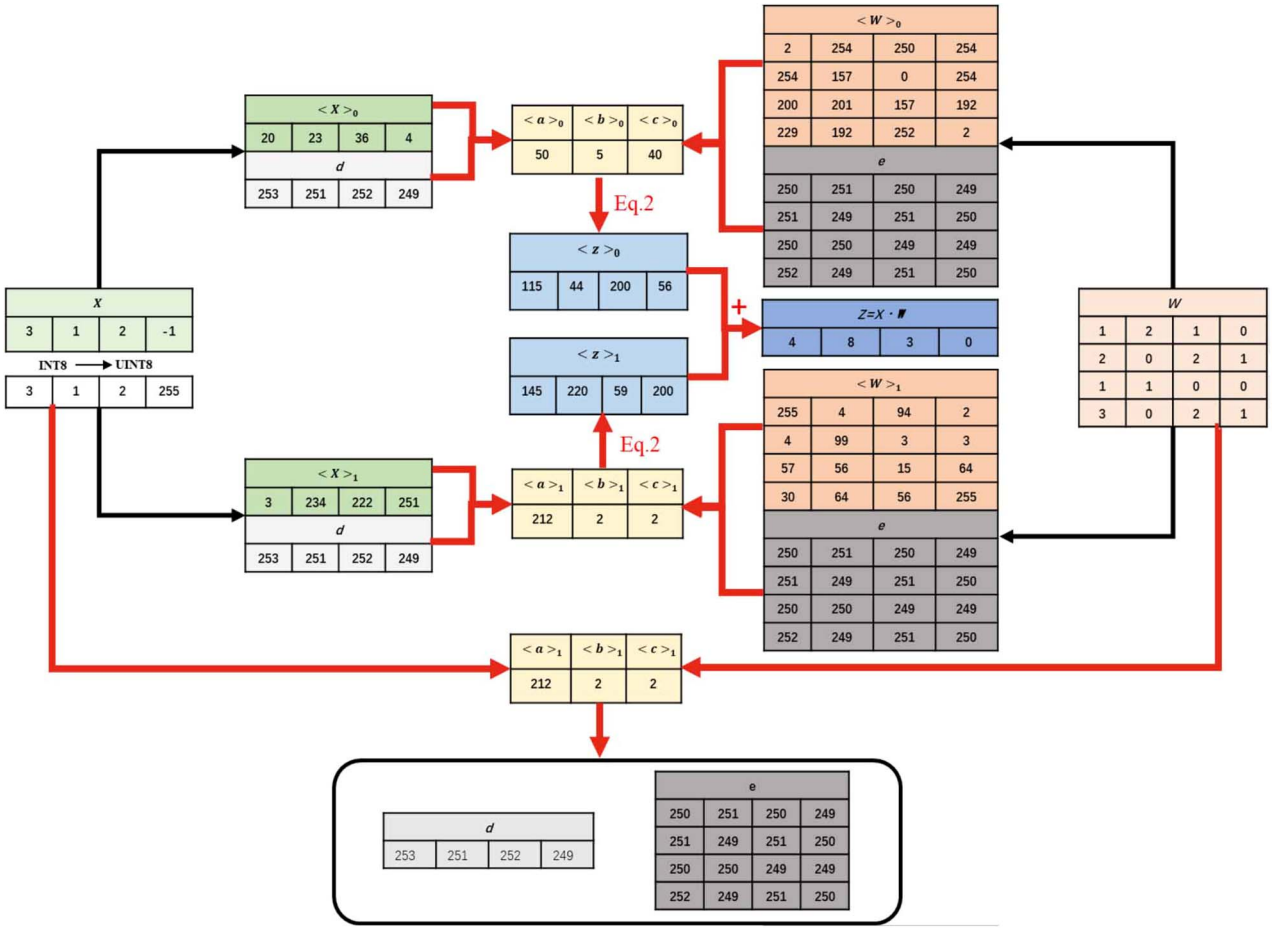


Figure 3. Schematic of matrix multiplication (a 1 × 4 input vector X and a 4 × 4 weight matrix W).

significant bits, x_7 and x_6, is achieved through (1, 2)-OT, while the remaining bits are correlated using (1, 4)-OT.

Next, M_1 is defined as a $V \times U$ two-dimensional matrix, where V represents the number of shares obtained through the A2B conversion, and U corresponds to the number of groups resulting from the aforementioned A2B partitioning. $M_1(v, u)$ denotes the u^{th} group within the v^{th} share, where $v \in [0, V-1]$ and $u \in [0, U-1]$. We establish a randomly generated list of non-repetitive element labels with a length of L. In this context, the query is performed using an injective (but not necessarily surjective) function $\mathcal{F}(x)$. Additionally, M_0 is defined as a $V \times U \times L$ matrix. $M_0(v, u, l)$ represents an element, where $v \in [0, V-1]$, $u \in [0, U-1]$, and $l \in [0, L-1]$. Each 8-bit integer value, after undergoing A2B conversion, can be divided into five groups: $M_0(v, u, l)$, where $u \in \{0, 1, 2, 3, 4\}$. Each group has multiple potential comparison values, denoted as s. For example, $M_1(v, 3)$ has four comparison values: 00, 01, 10, and 11. The comparison results are illustrated in Equation (2).

$$M_0(v,u,l) = \begin{cases} 1, & M_1(v,u) < s \\ 2, & M_1(v,u) = s \\ 3, & M_1(v,u) > s \end{cases} \tag{2}$$

Figure 4 illustrates the relationship between the M_0 and M_1 matrices for the 8-bit data type. Figure 5 provides an example using an 8-bit integer (88) to clarify the definitions of M_0 and M_1.

M_0 \ M_1 / s	$M_1(v,0)$	$M_1(v,1)$	M_0 \ M_1 / s	$M_1(v,2)$	$M_1(v,3)$	$M_1(v,4)$
	x_7	x_6		$x_5\ x_4$	$x_3\ x_2$	$x_1\ x_0$
0	$M_0(v,0,0)$	$M_0(v,1,0)$	00	$M_0(v,2,0)$	$M_0(v,3,0)$	$M_0(v,4,0)$
1	$M_0(v,0,1)$	$M_0(v,1,1)$	01	$M_0(v,2,1)$	$M_0(v,3,1)$	$M_0(v,4,1)$
			10	$M_0(v,2,2)$	$M_0(v,3,2)$	$M_0(v,4,2)$
			11	$M_0(v,2,3)$	$M_0(v,3,3)$	$M_0(v,4,3)$

Figure 4. A possible value comparison matrix with 8-bit data.

INT8(88)						
M_0 \ M_1 / s	0	1	M_0 \ M_1 / s	01	10	00
0	2	3	00	3	3	2
1	1	2	01	2	3	1
			10	1	2	1
			11	1	1	1

$Enc(M_0)$

200	70	18	118
70	231	96	210
185	250	66	233
30	120	14	176

Figure 5. Message generation, encryption, packaging.

Algorithm 4 provides a comprehensive description of our secure comparison protocol. Upon message recovery, an additional step is undertaken to pack the encrypted messages into

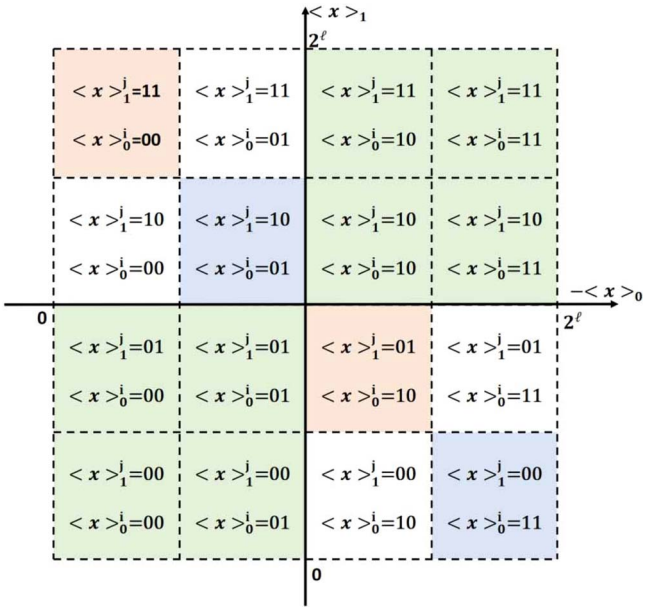


Figure 6. The region map is partitioned into three distinct color-coded areas, namely green, red, and blue, based on the most significant two bits of $\langle x\rangle_0$ and $\langle x\rangle_1$.

matrices. For instance, in Fig. 6, each INT8 value is encapsulated within a UINT8 4×4 matrix. Thus, the potential value comparison matrix transforms an 8-bit value into an 8-bit 4×4 matrix. For a value of ℓ-bits, where $U=\lfloor \ell/2\rfloor+1$, the size of the packing matrix is $\lceil \ell/2\rceil\times4$. As depicted in Fig. 6 ($\ell=8, U=5$), the leftmost two groups each have two possible values, while the rightmost three groups each have four possible values. Consequently, the data from the leftmost two groups can be efficiently combined, and a 4×4 matrix can be utilized for data packing.

Algorithm 4 Secure comparison

Input: S_0 and S_1 hold the group number G, and a list of non-repeating, randomly generated elements *label*, with a length of L.

Output: the comparison result T_m.

1: S_0 locally generates a random number r_0, computes $\hat{r_0}=G^{r_0} \bmod 2^\ell$, and sends $\hat{r_0}$ to S_1.

2: S_1 receives $\hat{r_0}$, locally generates a list of random numbers r_0 of length U, and creates a two-dimensional matrix $R=\left(\hat{r}_0^{\mathcal{F}(M_1)} \bmod 2^\ell\right)\oplus\left(G^{r_1} \bmod 2^\ell\right)$, where M_1 is a $V\times U$ matrix. Then S_1 sends R to S_0.

3: S_0 receives R and generates $key_0=R\oplus\left(\left(\hat{r}_0^{\mathcal{F}(M_0)} \bmod 2^\ell\right)^{r_0} \bmod 2^\ell\right)$. Then S_0 obtains $Enc(M_0)=M_0\oplus key_0$ by encrypting M_0 with key_0, where M_0 is a $V\times U\times L$ matrix. Finally, S_0 sends $Enc(M_0)$ to S_1.

4: S_1 receives $Enc(M_0)$ and generates $key_1=\hat{r}_0^{r_0 \bmod 2^\ell}$. Then S_1 decrypts $Enc(M_0)$ using key_1 and generates the T_m matrix, which is the comparison result.

5.4.2. *Secure ReLU computation*

While performing the $ReLU=max(0,x)$ operation is straightforward in plaintext, it becomes complex and challenging when applied to encrypted data. Previous approaches integrate secret sharing from one party, which includes the data's sign, e.g. GC. However, in methods based on AdSS, the impact of sign bits and modular operations cannot be ignored. Therefore, directly applying conventional secure comparison methods [16] is infeasible.

For instance, consider the case where $(-\langle x\rangle_0, \langle x\rangle_1)=(-90,3)$ in INT8 binary representation. A traditional secure comparison would incorrectly conclude $-90>3$ due to their respective binary representations as $(10100110)^b$ and $(00000011)^b$. In reality, for 2PC ReLU, the comparison should be between $(\langle x\rangle_0+\langle x\rangle_1) \bmod 2^\ell$ and 0, rather than between $-\langle x\rangle_0$ and $\langle x\rangle_1$.

In the context of plaintext quantized CNN, the inputs to the ReLU function are encoded as ℓ-bit signed numbers using the 2's complement representation. As shown in Fig. 6, we utilize $-\langle x\rangle_0$ and $\langle x\rangle_1$ as the x and y axes, respectively. We also employ the two most significant bits of $-\langle x\rangle_0$ and $\langle x\rangle_1$ (denoted as $\langle x\rangle_0^i$ and $\langle x\rangle_1^j$) to infer the position where x falls. Additionally, we explore the possible resulting values of $(\langle x\rangle_0+\langle x\rangle_1) \bmod 2^\ell$ based on the range of $-\langle x\rangle_0$ and $\langle x\rangle_1$.

When x falls within the green region, we can directly compare the magnitudes of $-\langle x\rangle_0$ and $\langle x\rangle_1$ to determine the sign. For x falling outside the green area, the sign is based on the possible resulting values of $(\langle x\rangle_0+\langle x\rangle_1) \bmod 2^\ell$. The sign is considered negative in the red region and positive in the blue region. Consequently, when x falls within the uncolored region, its sign cannot be directly determined. To address this issue comprehensively, we combine this coordinate system with our secure comparison protocol, resulting in an A2B-based secure ReLU computation approach.

We illustrate the workflow of our secure ReLU computation using the example of $(\langle x\rangle_0, \langle x\rangle_1)=(120,8)$. For the values $-\langle x\rangle_0$ and $\langle x\rangle_1$, where $-\langle x\rangle_0=-120$. After applying the A2B transformation, it becomes $-\langle x\rangle_0 \leftarrow 1 \parallel 0 \parallel 00 \parallel 10 \parallel 00$, and $\langle x\rangle_1 \leftarrow 0 \parallel 0 \parallel 00 \parallel 10 \parallel 00$. By comparing the first two groups, i.e. the two most significant groups, we can observe that x falls within the uncolored region. Therefore, it is necessary to proceed with our secure comparison protocol for comparing the remaining groups from left to right to further determine the sign. The result indicates the comparison between $-\langle x\rangle_0$ and $\langle x\rangle_1$. Based on this outcome, we conclude that $x<0$. By reconstructing $rec((\langle x\rangle_0+\langle x\rangle_1) \bmod 2^\ell)=-128$, we can verify the correctness of our proposed secure ReLU computation.

5.5. Communication complexity analysis

5.5.1. *Complexity analysis of linear computation*

In this section, we analyze the communication complexity involved in the secure linear computations of F2PQNN. As shown in Table 2, we benchmark the communication complexity of the linear layer functions against MiniONN [17], Sonic [9], Pio [33], and Cenia [10]. Let n represent the size of the feature map, and t is the number of matrix inner products in a convolution operation. As discussed previously, during linear computations, F2PQNN utilizes algorithms of Triplet Generation, Secure Matrix Multiplication, and Secure Truncation, resulting in communication complexities of $\mathcal{O}(n/t)$, $\mathcal{O}(n/t)$, and $\mathcal{O}(n/t)$, respectively.

5.5.2. *Complexity analysis of nonlinear computation*

In this section, we analyze the communication complexity associated with the secure nonlinear computations of F2PQNN, as detailed in Table 3. Specifically, there are three types of secure comparison protocols: GC-based protocols (e.g. MiniONN [17] and Pio [33]), BSS-based protocols (e.g. Sonic [9]), and AdSS-based protocols (e.g. Cenia [10] and proposed F2PQNN). Secure comparison protocols based on GC and BSS use bitwise operations for comparisons, resulting in a complexity dependent on ℓ. Sonic employs

Table 2. Communication complexities of secure linear computation

Function	Benchmarks	Comp.
Triplet generation	MiniONN[17]	$\mathcal{O}(n/t)$
	Sonic[9]	$\mathcal{O}(nt)$
	Pio[33]	$\mathcal{O}(nt)$
	Cenia[10]	–
	F2PQNN	$\mathcal{O}(n/t)$
Secure matrix multiplication	MiniONN[17]	$\mathcal{O}(n/t)$
	Sonic[9]	$\mathcal{O}(nt)$
	Pio[10]	$\mathcal{O}(nt)$
	Cenia[33]	–
	F2PQNN	$\mathcal{O}(n/t)$

Table 3. Communication complexities of secure nonlinear computation

Function	Benchmarks	Comp.
Secure comparison	MiniONN[17]	$\mathcal{O}(\lambda\ell)$
	Sonic[9]	$\mathcal{O}(\log \ell)$
	Pio[33]	$\mathcal{O}(\lambda\ell)$
	Cenia[10]	$\mathcal{O}(1)$
	F2PQNN	$\mathcal{O}(1)$

a tree structure, reducing the computational complexity from $\mathcal{O}(\ell)$ to $\mathcal{O}(\log \ell)$. In contrast, AdSS-based protocols use arithmetic circuits for comparisons, eliminating the need for complex bitwise operations and achieving $\mathcal{O}(1)$ complexity. It is comparable to that of plaintext comparison functions. Here, ℓ is a security parameter in GC, which is typically set to 128. λ represents the bit width of an element in $\mathbb{Z}_{2^\ell}$.

6. PROBABILISTIC ROUNDING ANALYSIS

In this section, we analyze the error of secure matrix multiplication under quantization. The analysis demonstrates that our proposed scheme introduces acceptable errors, and is effective for secure predictions in neural networks.

In Section 5.2, the QNN is introduced. Thus, for any $x \in \mathbb{R}$, it can be quantized into an integer $Q^f(x) := \lfloor x \cdot 2^f \rceil$. We calculate the error as follows: $\left|x - Q^f(x)2^{-f}\right| = 2^{-f}\left|x \cdot 2^f - Q^f(x)\right| \le 2^{-f-1} = \epsilon/2$, where $\epsilon := 2^{-f}$ is the smallest positive number. The integer representation of the product xy is denoted by $O^f(xy)$. Its definition is as follows:

$$\begin{aligned} O^f(xy) &:= \lfloor \zeta \rfloor + e \\ \zeta &:= Q^f(x)Q^f(y)2^{-f}, \quad e \sim \mathfrak{B}(\{\zeta\}), \end{aligned} \tag{3}$$

where $\{\zeta\}$ is the decimal part of ζ ($0 \le \{\zeta\} < 1$). The random variable $0 \le \{\zeta\} < 1$ represents rounding to the lower or higher value, and $\mathfrak{B}(\cdot)$ denotes the Bernoulli distribution with a given mean. The inputs $Q^f(x)$ and $Q^f(y)$, as well as the output $O^f(xy)$, are all integers. To obtain the fundamental fixed points, all of them must be multiplied by 2^{-f}. The Bernoulli random variable e in equation (3) is independently sampled for each computed product.

Given two real matrices $M_{a\times b}$ and $N_{b\times c}$ denoted as $Q^f(M)$ and $Q^f(N)$, where $Q^f(\cdot)$ represents any matrix with integer representation, the product of their integer representations is denoted as $O^f(MN) := (\sum_l O^f(M_{il}N_{lj}))_{a\times c}$. The following proposition guarantees its correctness.

Proposition 6.1. $\forall M \in \mathbb{R}^{a\times b}, \forall N \in \mathbb{R}^{b\times c}, \mathcal{E}\left(O^f(MN)\right) = Q^f(M) \cdot Q^f(N)\, 2^{-f}$, the symbol $\mathcal{E}(\cdot)$ represents the expectation concerning the random rounding in the equation (3).

It can be observed that the expected value of $O^f(MN)$ is exactly equal to the product of matrices $Q^f(M)$ and $Q^f(N)$ rounded up. Therefore our secure matrix multiplication is unbiased. If the product of the quantized numbers is replaced by the nearest rounding, $\tilde{O}^f(xy) := \lfloor Q^f(x)\, Q^f(y)\, 2^{-f} \rceil$, equation (3) is replaced. The matrix product obtained through nearest rounding is biased, as it is only unbiased when the rounding error is exactly zero, which is typically not the case.

To quantify the error during the execution of secure matrix multiplication, we define the worst-case error as follows:

Proposition 6.2. Suppose $M \in \mathbb{R}^{a\times b}$, and $N \in \mathbb{R}^{b\times c}$, with the absolute values of all entries bounded by $2^k (k \ge 0)$. Then $\left\|O^f(MN) - 2^f MN\right\| < \sqrt{ac} \cdot b \cdot \left(2^k + 1 + \epsilon/4\right)$, where $\epsilon := 2^{-f}$, $\|\cdot\|$ denotes the Frobenius norm.

In Proposition 6.2, the parameter k corresponds to the number of bits used to represent the integer part of any $x \in \mathbb{R}$, then we have the trivial bound $\|MN\| \le \sqrt{ac} \cdot b \cdot 2^{k+1}$. Therefore, the error $\left\|2^{-f}O^f(MN) - MN\right\|$ calculated in Proposition 6.2 is small and acceptable. To avoid large errors, the accuracy parameter f should be sufficient high.

The proposition presented below demonstrates that the rounding method employed in equation (3) adheres to a probability bound, which represents an improvement over the aforementioned worst-case bound.

Proposition 6.3. Suppose $M \in \mathbb{R}^{a\times b}, N \in \mathbb{R}^{b\times c}$, if the probability is at least $1 - \frac{1}{4\iota^2}$, then the following holds: $\|O^f(MN) - 2^{-f} \cdot Q^f(M)\, Q^f(N)\| \le \iota\sqrt{abc}$.

According to Proposition 6.1, $2^{-f}Q^f(M)\,Q^f(N)$ represents the expectation of $O^f(MN)$. Therefore, the first term on the left side of inequality in Proposition 6.3 represents the distance between $O^f(MN)$ and its expectation. We can compare Proposition 6.3 with Proposition 6.2 by taking the expectation as $2^{-f}\ Q^f(M)\ Q^f(N) \approx 2^f(MN)$. Consequently, our matrix multiplication achieves higher accuracy compared to alternative implementations that rely on nearest rounding, particularly in the case of dealing with large matrices (i.e. b is large).

7. SECURITY ANALYSIS

In our solution, the user input, the QNN model (i.e. weights), and the intermediate results are securely encrypted as secret shares using AdSS and OT techniques. By utilizing this approach, the confidentiality of sensitive information is effectively preserved. Throughout the execution process, both queries and the model itself are randomly partitioned and distributed among two non-colluding servers, ensuring that no private data are leaked. To provide further details, our solution leverages AdSS and OT to perform nonlinear activations on the non-colluding servers. These servers do not possess any prior knowledge of the user's private input data, as it is securely processed using precomputed triples.

Consequently, no meaningful information is disclosed to either server, thus guaranteeing the utmost security.

To establish a rigorous security foundation, we define an ideal functionality, denoted as $\mathfrak{F}^{\mathcal{ONT}}$, as the desired target for outsourced neural network inference. We then present a formal security definition based on this ideal functionality and provide a proof demonstrating that this functionality is securely achieved within an ideal/real-world simulation paradigm.

The ideal functionality $\mathfrak{F}^{\mathcal{ONT}}$ encompasses the security properties for an inference protocol involving the outsourcing of a neural network.

Definition 1. The ideal functionality $\mathfrak{F}^{\mathcal{ONT}}$, pertaining to outsourced neural network inference in the cloud, includes the following key components:

Input: The pre-trained QNN model parameters $\mathcal{M}$ are transmitted by the model owner $\mathcal{O}$, while $\mathcal{U}$ provides the data $\mathcal{X}$. S_0 and S_1 do not contribute any inputs in this context.

Computation: After receiving both the model parameters $\mathcal{M}$ and the data $\mathcal{X}$, $\mathfrak{F}^{\mathcal{ONT}}$ performs inference by employing ideal functionalities, ultimately producing the desired inference result.

Output: The inference result is delivered back to the user by $\mathfrak{F}^{\mathcal{ONT}}$, completing the outsourced neural network inference process.

The security guarantees are established in the following manner:

Definition 2. Protocol Π is deemed secure for evaluating the functionality $\mathfrak{F}^{\mathcal{ONT}}$ when faced with passive adversaries under fixed corruption conditions. If there exists a PPT adversary $\mathcal{A}$ targeting the real model, and a corresponding PPT simulator $\hat{\mathcal{S}}$ for the ideal model. This simulator $\hat{\mathcal{S}}$ guarantees that the security requirements are satisfied for all parties $\mathcal{P}$ within the set $[\mathcal{O}, \mathcal{U}, S_0, S_1]$. Specifically,

$$\{I_{\mathfrak{F}^{\mathcal{ONT}},\mathcal{P},\hat{\mathcal{S}}}(\mathcal{X},\mathcal{M})\}_{\mathcal{X},\mathcal{M}} \equiv_c \{R_{\Pi,\mathcal{P},\mathcal{A}}(\mathcal{X},\mathcal{M})\}_{\mathcal{X},\mathcal{M}}.$$

Here, $I_{\mathfrak{F}^{\mathcal{ONT}},\mathcal{P},\hat{\mathcal{S}}}()$ represents the output of the joint execution of the $\mathfrak{F}^{\mathcal{ONT}}$ functionality in the ideal world, and $R_{\Pi,\mathcal{P},\mathcal{A}}()$ represents the outcome of the protocol Π in the real world.

The following scenarios are examined:

- **Corrupted model owner.** In the case of a corrupted model owner, it is crucial to ensure that no sensitive information related to the user input $\mathcal{X}$ is revealed or disclosed. To achieve this, the corrupted model owner $\mathcal{O}$ interacts with a simulator $\hat{\mathcal{S}}$, which retrieves the model input supplied by the adversary, and transmits it securely to the semi-honest parties S_0 and S_1. Importantly, the model owner $\mathcal{O}$ should not be able to distinguish between the transcripts resulting from $\hat{\mathcal{S}}$ in the ideal scenario of the $\mathfrak{F}^{\mathcal{ONT}}$ functionality and the real scenario of the protocol Π.
- **Corrupted user.** When a mobile user is corrupted, it becomes imperative to ensure that no confidential information about the model weights and coefficients $\mathcal{M}$ is disclosed or obtained, except for the publicly available hyperparameters. To achieve this, a simulator $\hat{\mathcal{S}}$ interactions with the $\mathcal{U}$, extracting the user input $\mathcal{X}$ and securely transmitting it to S_0 and S_1. $\hat{\mathcal{S}}$ also emulates the output z. Importantly, $\mathcal{U}$ should not be able to differentiate between the transcripts resulting from interactions with $\hat{\mathcal{S}}$ in the ideal scenario of the $\mathfrak{F}^{\mathcal{ONT}}$ functionality and the real scenario of the protocol Π.
- **Corrupted cloud server**. In the event of a compromised cloud server S_i, it is essential to prevent the server from acquiring any private information regarding the user input $\mathcal{X}$, model weights, and coefficients $\mathcal{M}$, except for the publicly available hyperparameters. To achieve this, a simulator $\hat{\mathcal{S}}$ interacts with the corrupted server S_i and precisely emulates the transcripts for interactions between the adversary S_i and the honest parties $\mathcal{O}$, $\mathcal{U}$, S_0 or S_1. Importantly, the corrupted cloud server S_i should be unable to distinguish between the transcripts resulting from interactions with the simulator $\hat{\mathcal{S}}$ in the ideal scenario of the $\mathfrak{F}^{\mathcal{ONT}}$ functionality and the real scenario of the protocol Π.

Theorem 1. Based on Definition 2, it can be concluded that our protocol effectively and securely implements $\mathfrak{F}^{\mathcal{ONT}}$ in the context of a semi-honest adversary $\mathcal{A}$.

***Proof* 1.** To begin, we establish the definition of the simulator $\hat{\mathcal{S}}$, which is constructed based on the corrupted party. The objective of the simulator is to faithfully replicate the transcripts, including the joint distribution of inputs and outputs, in a manner that is indistinguishable from the observed transcripts in the ideal world execution, with respect to computational capabilities. In essence, the simulated transcripts should closely resemble the actual interactions, ensuring that the semi-honest parties correctly acquire the expected outputs.

- **Simulator for the corrupted model owner:** In the protocol Π, the corrupted model owner $\mathcal{O}$'s only task is to provide the $\mathcal{M}$ as input. In the ideal world, $\hat{\mathcal{S}}$ assumes the role of the $\mathcal{O}$ by submitting the model $\mathcal{M}$ to the ideal functionality $\mathfrak{F}^{\mathcal{ONT}}$. The honest cloud servers then execute the remaining protocol Π by sequentially combining the ideal functionalities of subroutines. Upon completion of Π, only the shares of the correct result z are returned to the honest user $\mathcal{U}$ to ensure correctness, while no output is provided to $\mathcal{O}$. To simulate $\mathcal{O}$'s perspective, the simulator $\hat{\mathcal{S}}$ generates a dummy representation of $\mathcal{M}$ as its output. The output of the simulator $\hat{\mathcal{S}}$, denoted as $\hat{\mathcal{S}}(\mathcal{M})$, is distributed identically to $\mathcal{O}$'s view $View^{\Pi}_{\mathcal{U}}(\mathcal{M})$. Consequently, the difference between the real world and the ideal world is negligible and can be ignored.
- **Simulator for the corrupted user**: In the protocol Π, the corrupted user $\mathcal{U}$ participates by submitting the secret shares of the user input $\mathcal{X}$ and receiving the two shares $\langle z\rangle_0$ and $\langle z\rangle_1$ of the inference result z. In the ideal world, the simulator $\hat{\mathcal{S}}$ assumes the role of the corrupted $\mathcal{U}$ by submitting the user input $\mathcal{X}$ to the $\mathfrak{F}^{\mathcal{ONT}}$ functionality. After receiving the result z from $\mathfrak{F}^{\mathcal{ONT}}$, $\hat{\mathcal{S}}$ generates a random value $\langle z'\rangle_0$ chosen from $\mathbb{Z}_{2^\ell}$ and computes $\langle z'\rangle_1 = z - \langle z'\rangle_0$. Then, $\hat{\mathcal{S}}$ outputs $\langle z'\rangle_0$ and $\langle z'\rangle_1$. The security is guaranteed as the additive secret shares $\langle z'\rangle_0$ and $\langle z'\rangle_1$ are uniformly random values in $\mathbb{Z}_{2^\ell}$, and are indistinguishable from the transcripts in the real world. Moreover, the result z can be reconstructed by computing $\langle z'\rangle_0 + \langle z'\rangle_1 \pmod{2^\ell}$. The output $\hat{\mathcal{S}}(\mathcal{X}, z)$ is indistinguishable from the view of $\mathcal{U}$, represented as $View^{\Pi}_{\mathcal{U}}(\mathcal{X}, z)$. Consequently, the difference between the real world and the ideal world is negligible and can be ignored.
- **Simulator for a corrupted cloud server:** In the protocol Π, the two cloud servers S_0 and S_1 are not required to submit inputs or receive the final result. Computation and interaction between them occur through the use of secret shares at various stages of Π. Given their symmetrical nature, it is

sufficient to create a simulator $\hat{\mathcal{S}}$ for one of the servers, e.g. S_0. In this protocol, $\hat{\mathcal{S}}$ systematically combines the simulators of subroutines to replicate the behavior of the honest cloud server S_1, thereby interacting with S_0. The messages exchanged in the subroutines consist of random shares of matrices with dimensions consistent on the shares of the semi-honest server. The interactions between the two servers are emulated by utilizing the simulator $\hat{\mathcal{S}}^{BTri}$ for Beaver's multiplication $\mathfrak{F}^{BTri}$. This simulator interacts with S_0 by employing random shares.

Furthermore, the perspective of S_0 during each interaction within Π cannot be distinguished from the perspective emulated by $\hat{\mathcal{S}}^{BTri}$. As a result, we assert that the simulator $\hat{\mathcal{S}}$ in the hybrid model can faithfully emulate the perspective of S_0, thereby ensuring that the two worlds are indistinguishable. This completes the proof of Theorem 1.

8. EXPERIMENTAL EVALUATION

8.1. System setup and model architecture

We implement our solution utilizing Python bindings within a C++ framework. All experiments are conducted on a CentOS 7.9 server equipped with an AMD EPYC 7552 CPU processor (@2.60GHz) and 64GB DDR4 RAM, which supports AES-NI. To simulate two parties, we employ two distinct terminal ports. Following the MiniONN protocol, we utilize YASHE for fully homomorphic encryption and implemented it using the SIMD-enabled SEAL library [49]. Our experimental evaluation is performed on four widely used and realistic datasets: MNIST, Fashion-MNIST, STL-10, and CIFAR-10. The MNIST dataset comprises 60 000 grayscale images of handwritten digits, each sized at 28 × 28 pixels and categorized into 10 distinct classes. Fashion-MNIST comprises 70 000 grayscale images of fashion products, also sized at 28 × 28 pixels and covering 10 categories. STL-10 contains 13 000 color images from 10 categories, each measuring 96 × 96 pixels. CIFAR-10 consists of 60 000 color images of size 32 × 32 pixels, divided into 10 different classes. For training purposes, 50 000 images from both MNIST and CIFAR-10 are designated for training, while the remaining 10 000 images are reserved for testing. In Fashion-MNIST, 60 000 images are allocated for training, with 10 000 images set aside for testing. In STL-10, 5000 images are used for training, and the remaining 8000 images are utilized for testing.

In addition to dataset evaluation, it is crucial to demonstrate the performance of our system using actual neural network models. We employed two QNN architectures, named Network 1 and Network 2, following the MiniONN [17] and EzPC [50] frameworks, respectively. Detailed descriptions of these architectures can be found in Table 4 (MNIST, Fashion-MNIST) and Table 5 (STL-10, CIFAR-10).

Consistent with existing techniques [17], [18], we evaluate our system's performance in a LAN setting. The secure neural network inference is conducted within a secret shared domain, where all data are represented as fixed-point integers and shared over a ring $\mathbb{Z}_{2^\ell}$. In our experiments, the ring size is set to $\mathbb{Z}_{2^{32}}$, aligning with previous works [14].

We perform simulation experiments to evaluate the impact of different quantization precisions on accuracy and loss for Network 1 and Network 2. As shown in Figures 7 and 8, model training is performed with the stochastic gradient descent optimizer, on a learning rate of 0.01 and a batch size of 256. These chosen parameters for the learning rate and batch size are based on satisfactory performance observed in plaintext training samples. We conduct training tests with three different quantization precisions: 4-bit quantization, 8-bit quantization, and 16-bit quantization. The experimental results indicate that model accuracy improves as training time increases, and stabilizes after 10 epochs.

On the MNIST dataset, after 50 epochs, the F2PQNN model can achieve an accuracy of 99.09% under 8-bit quantization, while the accuracy for both 4-bit and 16-bit quantization is approximately 98.6%. For the Fashion-MNIST dataset, after 50 epochs, the F2PQNN model can attain an accuracy of 91.01% under 8-bit quantization, with other quantization precisions yield around 90.5%. On the CIFAR-10 dataset, after 50 epochs, the F2PQNN model has achieved an accuracy of 78.7% under 8-bit quantization, while other quantization precisions yield around 70%. Additionally, for the STL-10 dataset, after 50 epochs, the accuracy under 8-bit quantization can reach 84.7%, while other quantization precisions reach only about 83%.

We also focus on the 8-bit quantization precision under probabilistic rounding, as it can offer the best overall performance in terms of accuracy and efficiency. The choice of 8-bit quantization is justified by its proximity to the lower limit of reasonable results. As Figs 7 and 8 demonstrated, increasing quantization precision does not significantly enhance model accuracy. For the MNIST, Fashion-MNIST, STL-10, and CIFAR-10 datasets, all images are quantized as integers within the range of [0, 255]. The quantization parameters for each layer are shared by the model owner across all servers. The weights are quantized into a uniform and symmetric 8-bit representation (INT8), while the biases are quantized into a 16-bit representation. Through this configuration, all user-input images can be directly fed into the cloud server for secure inference without the need for data preprocessing. Additionally, the quantization parameters are encoded into all layers, ensuring the seamless integration of the quantized model.

8.2. Different experimental evaluation metrics

F2PQNN (8-bit Quantized 2PC-CNN solution) is compared with the following approaches: MiniONN [17], EzPC [50], Sonic [9], Pio [33], and Cenia [10]. Specifically, we consider three crucial metrics: Execution Time (Time.), Accuracy (Acc.), and Communication Overhead (Comm.).

8.2.1. *Execution time analysis*

F2PQNN reduces execution time overhead through the collaborative efforts of quantization and offline stage encapsulation, as well as asynchronous computation of triplets.

As shown in Table 6, our triplet generation speed is several orders of magnitude faster than that of previous methods. By further incorporating asynchronous computation, it has achieved a 13.6% increase in efficiency, resulting in an overall speedup of 4697×.

The results show that the quantization model proposed in our framework demonstrates significant benefits for F2PQNN. Reducing the bit width by 2× can decrease the latency of F2PQNN by approximately twofold. The bit width directly affects the volume of communication data in F2PQNN. This is because in the matrix multiplication operations of SCONV, the primary overhead arises from computation, resulting in less noticeable latency improvements. Similarly, the enhancements for SFC are modest since this operation does not require communication and can be executed using AdSS. Consequently, our quantization model can reduce the execution time of SReLU by approximately 2×.

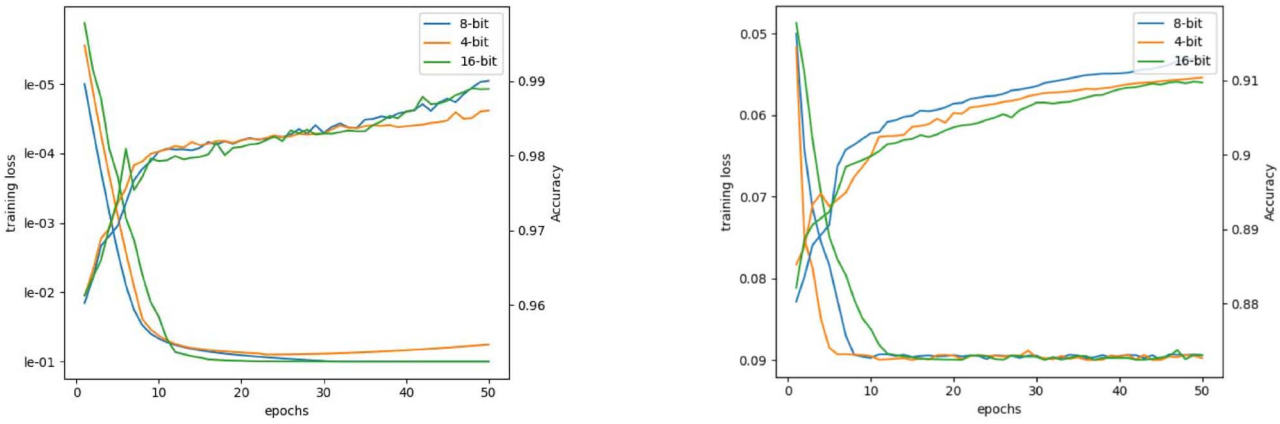
Table 7 reports the time overhead for each stage of Network 1 and Network 2. During the offline stage, the two servers interact to

Table 4. The NN architecture on MNIST and Fashion-MNIST dataset(Network 1)

Layerth	**Layer**	**Description**
1	*Convolution*	Input image 28 × 28, window size 5 × 5,stride(1,1), number of output channels of 16: $\mathbb{R}^{16\times576} \longleftarrow \mathbb{R}^{16\times25} \times \mathbb{R}^{25\times576}$
2	*ReLU activation*	Calculates ReLU for each input
3	*Max pooling*	Window size 1 × 2 × 2 and outputs $\mathbb{R}^{16\times12\times12}$
4	*Convolution*	Input image 12 × 12, window size 5 × 5,stride(1,1), number of output channels of 16: $\mathbb{R}^{16\times64} \longleftarrow \mathbb{R}^{16\times400} \times \mathbb{R}^{400\times64}$
5	*ReLU activation*	Calculates ReLU for each input
6	*Max pooling*	Window size 1 × 2 × 2 and outputs $\mathbb{R}^{16\times4\times4}$
7	*Fully connected*	Fully connects the incoming 256 notes to the outgoing 100 nodes: $\mathbb{R}^{100\times1} \longleftarrow \mathbb{R}^{100\times256} \times \mathbb{R}^{256\times1}$
8	*ReLU activation*	Calculates ReLU for each input
9	*Fully connected*	Fully connects the incoming 100 notes to the outgoing 10 nodes: $\mathbb{R}^{10\times1} \longleftarrow \mathbb{R}^{10\times100} \times \mathbb{R}^{100\times1}$

Table 5. The NN architecture on CIFAR-10 and STL-10 dataset(Network2)

Layerth	**Layer**	**Description**
1	*Convolution*	Input image 32 × 32, window size 3 × 3,stride(1,1), number of output channels of 64: $\mathbb{R}^{64\times1024} \longleftarrow \mathbb{R}^{64\times27} \times \mathbb{R}^{27\times1024}$
2	*ReLU activation*	Calculates ReLU for each input
3	*Convolution*	Window size 3 × 3,stride(1,1), number of output channels of 64: $\mathbb{R}^{64\times1024} \longleftarrow \mathbb{R}^{64\times576} \times \mathbb{R}^{576\times1024}$
4	*ReLU activation*	Calculates ReLU for each input
5	*Max pooling*	Window size 1 × 2 × 2 and outputs $\mathbb{R}^{64\times16\times16}$
6	*Convolution*	Window size 3 × 3,stride(1,1), number of output channels of 64: $\mathbb{R}^{64\times256} \longleftarrow \mathbb{R}^{64\times576} \times \mathbb{R}^{576\times256}$
7	*ReLU activation*	Calculates ReLU for each input
8	*Convolution*	Window size 3 × 3,stride(1,1), number of output channels of 64: $\mathbb{R}^{64\times256} \longleftarrow \mathbb{R}^{64\times576} \times \mathbb{R}^{576\times256}$
9	*ReLU activation*	Calculates ReLU for each input
10	*Max pooling*	Window size 1 × 2 × 2 and outputs $\mathbb{R}^{64\times8\times8}$
11	*Convolution*	Window size 3 × 3,stride(1,1), number of output channels of 64: $\mathbb{R}^{64\times64} \longleftarrow \mathbb{R}^{64\times576} \times \mathbb{R}^{576\times64}$
12	*ReLU activation*	Calculates ReLU for each input
13	*Convolution*	Window size 1 × 1,stride(1,1), number of output channels of 64: $\mathbb{R}^{64\times64} \longleftarrow \mathbb{R}^{64\times64} \times \mathbb{R}^{64\times64}$
14	*ReLU activation*	Calculates ReLU for each input
15	*Convolution*	Window size 1 × 1,stride(1,1), number of output channels of 64: $\mathbb{R}^{16\times64} \longleftarrow \mathbb{R}^{16\times64} \times \mathbb{R}^{64\times64}$
16	*ReLU activation*	Calculates ReLU for each input
17	*Fully connected*	Fully connects the incoming 1024 notes to the outgoing 10 nodes: $\mathbb{R}^{10\times1} \longleftarrow \mathbb{R}^{10\times1024} \times \mathbb{R}^{1024\times1}$

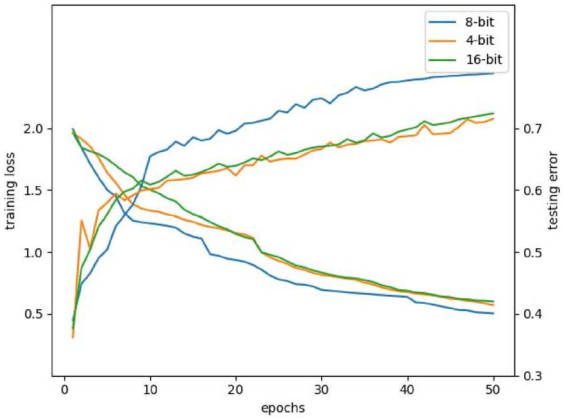


(a) Based on the MNIST dataset. (b) Based on the Fashion-MNIST dataset.

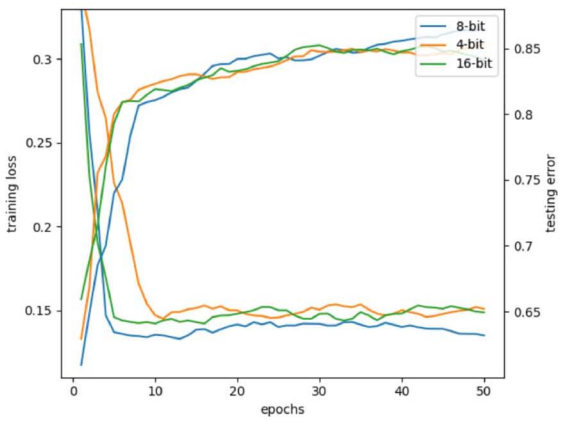
Figure 7. Different quantization precisions on the experimental accuracy and loss for Network 1.

generate triplets, with the process taking approximately 0.451s for the MNIST dataset, 0.571s for the Fashion-MNIST dataset, 8.038s for the CIFAR-10 dataset, and 3.125s for the STL-10 dataset. In the online stage, matrix multiplication, and activation functions dominate the time spent due to complex interaction computations. However, the execution time of the activation functions is reduced by approximately 2× due to shared execution across all layers and the utilization of quantization techniques.

Furthermore, most prior related works (such as MiniONN [17], EzPC [50], and Pio [33]) have considered client-server deployment scenarios, where private model weights and client inputs are concealed from the involved parties. The security protocols



(a) Based on the CIFAR-10 dataset.



(b) Based on the STL-10 dataset.

Figure 8. Different quantization precisions on the experimental accuracy and loss for Network 2.

Table 6. Triplet generation costs (ms)

Original triplet generation	Packed triplet generation	Packed triplet generation with Asyn. Comp.	Performance gain
79716.524	19.635	16.970	4697×

Table 7. Execution time for each stage on the different datasets

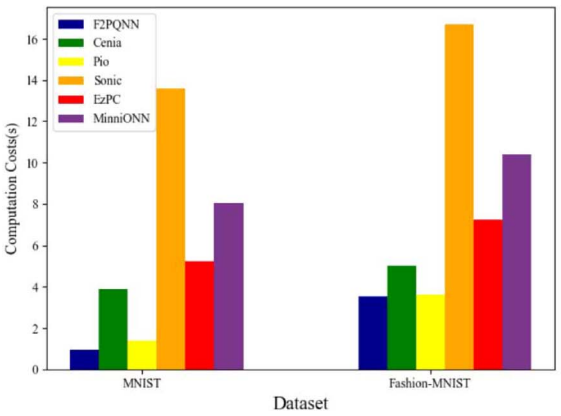
	MNIST / Fashion-MNIST		CIFAR-10 / STL-10			
Phases	**Stages**	**Time. (s)**	**Stages**	**Time. (s)**	**Stages**	**Time. (s)**
Offline	S_0	0.221 / 2.229	S_0	3.697 / 1.584	–	–
	S_1	0.230 / 0.262	S_1	4.341 / 1.541	–	–
Online	*Convolution*	0.056 / 0.055	*Convolution*	0.419 / 0.331	*Max pooling*	0.000 / 0.000
	ReLU activation	0.166 / 1.325	*ReLU activation*	10.478 / 4.816	*Convolution*	0.499 / 0.211
	Max pooling	0.001 / 0.002	*Convolution*	8.960 / 4.751	*ReLU activation*	0.598 / 0.298
	Convolution	0.096 / 0.098	*ReLU activation*	11.265 / 6.143	*Convolution*	0.068 / 0.007
	ReLU activation	0.147 / 1.217	*Max pooling*	0.001 / 0.001	*ReLU activation*	0.800 / 0.302
	Max pooling	0.000 / 0.000	*Convolution*	2.176 / 1.079	*Convolution*	0.011 / 0.006
	Fully connected	0.007 / 0.006	*ReLU activation*	3.000 / 1.550	*ReLU activation*	0.201 / 0.108
	ReLU activation	0.023 / 0.34	*Convolution*	1.998 / 0.898	*Fully connected*	0.001 / 0.001
	Fully connected	0.001 / 0.001	*ReLU activation*	3.026 / 1.860	–	–
Total	0.948 / 3.535		51.539 / 25.487			

in these methods do not require simultaneous operations on encrypted models and encrypted inputs. In contrast, secure outsourcing inference systems (including F2PQNN, Sonic [9], and Cenia [10]) require simultaneous protection of both the model and client input, i.e., secure protocols must be performed on both the model weights and client inputs in encrypted forms. Nevertheless, some of these works require multiple servers, which increases deployment complexity and overhead, thus F2PQNN shows superior efficiency compared to traditional client-server deployment approaches.

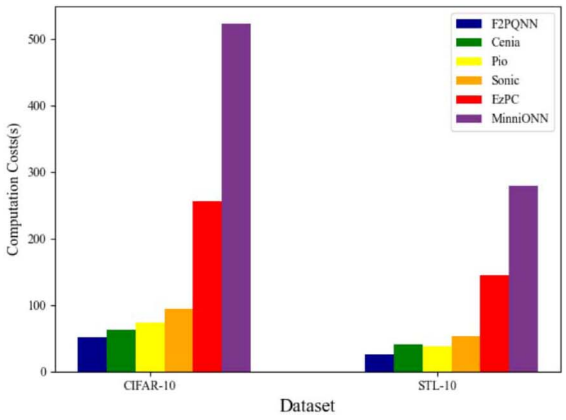
As shown in Fig. 9, the experimental results demonstrate a significant advantage of F2PQNN in execution time overhead across models of varying sizes. Specifically, on the MNIST dataset, F2PQNN achieves remarkable improvements in execution time overhead of 9.14×, 5.51×, 14.34×, 1.45×, and 4.08× compared to MiniONN [17], EzPC [50], Sonic [9], Pio [33], and Cenia [10], respectively. On the Fashion-MNIST dataset, F2PQNN shows notable improvements of 2.93×, 2.04×, 4.72×, 1.02×, and 1.42× compared to the same methods. On the CIFAR-10 dataset, F2PQNN achieves substantial improvements of 10.15×, 5×, 1.83×, 1.42×, and 1.21×, respectively. Finally, on the STL-10 dataset, F2PQNN delivers remarkable improvements of 10.97×, 5.69×, 2.08×, 1.5×, and 1.62×, compared to MiniONN [17], EzPC [50], Sonic [9], Pio [33], and Cenia [10], respectively.

8.2.2. *Communication overhead analysis*

Compared to SCONV, nonlinear layers such as SReLU and SMP require more computational resources to process input data of the same size. This increased demand arises from the additional arithmetic operations necessary to ensure data security. Consequently, the communication overhead primarily results from the computational requirements of the nonlinear layers. In contrast, the F2PQNN framework introduces a novel approach by



(a) computation overheads for Network 1.



(b) computation overheads for Network 2.

Figure 9. Comparison of the computation overheads.

combining secret sharing and OT techniques, particularly for handling activation functions within nonlinear layers, as a replacement for the commonly used obfuscation circuits. This integration leads to a substantial reduction in communication overhead compared to most existing approaches.

In this section, we compare our approach with MiniONN [17], EzPC [50], Sonic [9], Pio [33], and Cenia [10] in terms of communication overhead. Table 8 presents a comparison of the communicated data volume on Network 1 and Network 2 for each method. On the MNIST dataset (Network 1), F2PQNN achieves a reduction in communicated data volume ranging from 4.23× to 757.06×. On the Fashion-MNIST dataset (Network 1), the reduction ranges from 1.3× to 226×. On the CIFAR-10 dataset (Network 2), F2PQNN achieves a reduction from 1.62× to 6899.49×. On the STL-10 dataset (Network 2), the reduction ranges from 1.35× to 2485×. From these results, we observe that the communication overhead of EzPC [50] based on AdSS is greater than that of Sonic [9] based on BSS, and Cenia [10] based on AdSS. This discrepancy arises because participants in EzPC [50] incur input-related communication costs in each round to achieve constant-round communication. Building upon Cenia [10], we have meticulously designed SReLU, which integrates secret sharing and OT techniques. This design significantly reduces communication overhead and even surpasses that of BSS-based privacy-preserving neural network solutions.

Table 8. Communication overhead comparison of F2PQNN with methodology on different models

Model	Methodology	Comm.(MB)
Network 1 (MNIST)	MiniONN[17]	643.5
	EzPC[50]	471
	Sonic[9]	10.8
	Pio[33]	2.1
	Cenia[10]	3.60
	F2PQNN	0.85
Network 1 (Fashion-MNIST)	MiniONN[17]	701
	EzPC[50]	498
	Sonic[9]	17.1
	Pio[33]	4.14
	Cenia[10]	4.49
	F2PQNN	3.09
Network 2 (CIFAR-10)	MiniONN[17]	9070
	EzPC[50]	40500
	Sonic[9]	711
	Pio[33]	9.52
	Cenia[10]	22.5
	F2PQNN	5.87
Network 2 (STL-10)	MiniONN[17]	1631
	EzPC[50]	9695
	Sonic[9]	373
	Pio[33]	5.3
	Cenia[10]	17.3
	F2PQNN	3.9

8.2.3. *Accuracy analysis*

In Figs 7 and 8, we present a comprehensive analysis of F2PQNN's predictive accuracy and loss by exploring different levels of quantization precision. A comparative evaluation is performed against the corresponding plaintext predictions outlined in Table 9. Notably, F2PQNN achieves accuracy equivalent to the plaintext predictions, exhibiting a remarkable 99.09 and 91.01% accuracy for the Network 1 model on the MNIST dataset and Fashion-MNIST dataset. However, on the CIFAR-10 dataset and STL-10 dataset, F2PQNN achieves a secure inference accuracy of 78.7 and 84.7%, slightly lower than the plaintext prediction accuracy of 80.9 and 85.01% for the Network 2 model.

9. CONCLUSIONS AND FUTURE WORK

In this paper, we propose F2PQNN, a high-performance secure two-party inference service. It combines A2B sharing conversion and OT construction to implement ReLU. Additionally, F2PQNN adopts batching and asynchronous computation, effectively reducing communication overhead and significantly boosting inference speed. Furthermore, by employing quantization techniques, F2PQNN addresses the performance limitations encountered in current approaches. It can achieve this by generating lightweight 2PC-CNN models. At last, F2PQNN guarantees robust protection for both user inputs and trained model privacy. Rigorous evaluations adopt popular datasets and model architectures to demonstrate that F2PQNN outperforms existing solutions in terms of efficiency.

As part of future research, it is crucial to consider the potential risks from malicious cloud servers. Furthermore, we will consider model extraction attacks, model inversion attacks, and

Table 9. Inference accuracy (%) on various models for the MNIST, Fashion-MNIST, CIFAR-10, and STL-10 with proposed quantization

Model	F2PQNN	Plaintext
Network 1 (MNIST)	99.09%	99.09%
Network 1 (Fashion-MNIST)	91.01%	91.01%
Network 2 (CIFAR-10)	78.70%	80.90%
Network 2 (STL-10)	84.70%	85.01%

membership inference attacks in our subsequent work to provide a more comprehensive evaluation of our model's performance.

FUNDING

This work was supported by the National Natural Science Foundation of China (Grant No. 62372285).

CONFLICT OF INTEREST

None declared.

DATA AVAILABILITY

Data are openly available in a public repository. The data that support the findings of this study are openly available at http://deeplearning.net/data/mnist/, https://github.com/zalandoresearch/fashion-mnist, https://cs.stanford.edu/~acoates/stl10/ and http://www.cs.toronto.edu/~kriz/cifar.htm1.

ETHICAL APPROVAL

Not applicable.

REFERENCES

1. Boulemtafes A, Derhab A, Challal Y. A review of privacy-preserving techniques for deep learning. *Neurocomputing* 2020;**384**:21–45. https://doi.org/10.1016/j.neucom.2019.11.041.
2. Zheng Y, Wang C, Wang R. *et al.* Optimizing secure decision tree inference outsourcing. *IEEE Trans Dependable Secure Comput* 2023;**20**:3079–92. https://doi.org/10.1109/TDSC.2022.3194048.
3. Carlini N, Jagielski M, Mironov I. Cryptanalytic extraction of neural network models. In: *Annual International Cryptology Conference*. Santa Barbara, CA, USA: Springer, 2020, 189–218.
4. Dathathri R, Saarikivi O, Chen H. *et al.* CHET: an optimizing compiler for fully-homomorphic neural-network inferencing. In: *Proceedings of the 40th ACM SIGPLAN Conference on Programming Language Design and Implementation, Phoenix, AZ, USA*. New York: ACM, 2019, 142–56.
5. Hesamifard E, Takabi H, Ghasemi M. Deep neural networks classification over encrypted data. In: *Proceedings of the Ninth ACM Conference on Data and Application Security and Privacy, Richardson, Texas, USA*. New York: ACM, 2019, 97–108.
6. Zhang Q, Xin C, Wu H. GALA: greedy computation for linear algebra in privacy-preserved neural networks. In: *Proceedings 2021 Network and Distributed System Security Symposium*. Virginia: The Internet Society, 2021, 1–16.
7. Papernot N, Abadi M, Erlingsson Ú. *et al.* Semi-supervised knowledge transfer for deep learning from private training data. In: *5th International Conference on Learning Representations, ICLR 2017, Toulon, France, April 24–26, 2017, Conference Track Proceedings*. California: OpenReview.net, 2017, 1–16.
8. Bu Z, Dong J, Long Q. *et al.* Deep learning with Gaussian differential privacy. *Harv Data Sci Rev* 2020;**2020**:10–1162. https://doi.org/10.1162/99608f92.cfc5dd25.
9. Liu X, Zheng Y, Yuan X. *et al.* Securely outsourcing neural network inference to the cloud with lightweight techniques. *IEEE Trans Dependable Secure Comput* 2023;**20**:620–36. https://doi.org/10.1109/TDSC.2022.3141391.
10. Bi R, Xiong J, Luo C. *et al.* Communication-efficient privacy-preserving neural network inference via arithmetic secret sharing. *IEEE Trans Inf Forensics Secur* 2024;**19**:6722–37. https://doi.org/10.1109/TIFS.2024.3420216.
11. Wang J, He D, Castiglione A. *et al.* PCNNCEC: efficient and privacy-preserving convolutional neural network inference based on cloud-edge-client collaboration. *IEEE Trans Netw Sci Eng* 2023;**10**:2906–23. https://doi.org/10.1109/TNSE.2022.3177755.
12. Gilad-Bachrach R, Dowlin N, Laine K. *et al.* CryptoNets: applying neural networks to encrypted data with high throughput and accuracy. In: *International Conference on Machine Learning*. New York City, NY, USA: PMLR, 2016, 201–10.
13. Juvekar C, Vaikuntanathan V, Chandrakasan A. {GAZELLE}: a low latency framework for secure neural network inference. In: *27th USENIX Security Symposium (USENIX Security 18)*. Santa Clara, CA, USA: USENIX Association, 2018, 1651–69.
14. Mishra P, Lehmkuhl R, Srinivasan A. *et al.* DELPHI: a cryptographic inference system for neural networks. In: *Proceedings of the 2020 Workshop on Privacy-Preserving Machine Learning in Practice, Virtual Event USA*. New York: ACM, 2020, 27–30.
15. Rathee D, Rathee M, Kumar N. *et al.* CrypTFlow2: practical 2-party secure inference. In: *Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, Virtual Event USA*. New York: ACM, 2020, 325–42.
16. Demmler D, Schneider T, Zohner M. ABY-a framework for efficient mixed-protocol secure two-party computation. In: *NDSS, Dallas, Texas, USA*. California: The Internet Society, 2015, 6–8.
17. Liu J, Juuti M, Lu Y. *et al.* Oblivious neural network predictions via MiniONN transformations. In: *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*. New York: ACM, 2017, 619–31.
18. Riazi MS, Samragh M, Chen H. *et al.* {XONN}:{XNOR-based} oblivious deep neural network inference. In: *28th USENIX Security Symposium (USENIX Security 19), Santa Clara, CA, USA*. California: USENIX, 2019, 1501–18.
19. Jha NK, Ghodsi Z, Garg S. *et al.* DeepReDuce: ReLU reduction for fast private inference. In: *International Conference on Machine Learning*. New York: PMLR, 2021, 4839–49.
20. Ghodsi Z, Veldanda AK, Reagen B. *et al.* CryptoNas: private inference on a ReLU budget. *Adv Neural Inf Process Syst* 2020;**33**:16961–71.
21. Bi R, Xiong J, Li Q. *et al.* FPPNet: fast privacy-preserving neural network via three-party arithmetic secret sharing. In: *International Conference on Mobile Computing, Applications, and Services*. Berlin: Springer, 2022, 165–78.
22. Agrawal N, Shahin Shamsabadi A, Kusner MJ. *et al.* QUOTIENT: two-party secure neural network training and prediction. In: *Proceedings of the 2019 ACM SIGSAC Conference on Computer and Communications Security, London*. New York: ACM, 2019, 1231–47.

23. Dalskov A, Escudero D, Keller M. Secure evaluation of quantized neural networks. *Proc Priv Enhancing Technol* 2020;**2020**:355–75. https://doi.org/10.2478/popets-2020-0077.
24. Shen L, Dong Y, Fang B. *et al.* ABNN2: secure two-party arbitrary-bitwidth quantized neural network predictions. In: *Proceedings of the 59th ACM/IEEE Design Automation Conference, San Francisco, California*. New York: ACM, 2022, 361–6.
25. Luo Y, Xu N, Peng H. *et al.* AQ2PNN: enabling two-party privacy-preserving deep neural network inference with adaptive quantization. In: *2023 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. New York: ACM, 2023, 628–40.
26. Beaver D. Efficient multiparty protocols using circuit randomization. In: *Advances in Cryptology—CRYPTO'91: Proceedings 11*. Berlin: Springer, 1992, 420–32.
27. Smart NP, Vercauteren F. Fully homomorphic SIMD operations. *Des Codes Cryptogr* 2014;**71**:57–81. https://doi.org/10.1007/s10623-012-9720-4.
28. Vaidya J, Kantarcioğlu M, Clifton C. Privacy-preserving naive Bayes classification. *VLDB J* 2008;**17**:879–98. https://doi.org/10.1007/s00778-006-0041-y.
29. Vaidya J, Clifton C. Privacy preserving association rule mining in vertically partitioned data. In: *Proceedings of the Eighth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, Edmonton, Alberta, Canada*. New York: ACM, 2002, 639–44.
30. Agrawal R, Srikant R. Privacy-preserving data mining. In: *Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, Dallas, Texas, USA*. New York: ACM, 2000, 439–50.
31. Mohassel P, Zhang Y. SecureML: a system for scalable privacy-preserving machine learning. In: *2017 IEEE Symposium on Security and Privacy (SP)*. California: IEEE, 2017, 19–38.
32. Rouhani BD, Riazi MS, Koushanfar F. Deepsecure: Scalable provably-secure deep learning. In: *Proceedings of the 55th Annual Design Automation Conference, San Francisco California*. New York: ACM, 2018, 1–6.
33. Yang X, Chen J, He K. *et al.* Efficient privacy-preserving inference outsourcing for convolutional neural networks. *IEEE Trans Inf Forensics Secur* 2023;**18**:4815–29. https://doi.org/10.1109/TIFS.2023.3287072.
34. Riazi MS, Weinert C, Tkachenko O. *et al.* Chameleon: a hybrid secure computation framework for machine learning applications. In: *Proceedings of the 2018 on Asia Conference on Computer and Communications Security*. New York: ACM, 2018, 707–21.
35. Wagh S, Gupta D, Chandran N. SecureNN: 3-party secure computation for neural network training. *Proc Priv Enhancing Technol* 2019;**2019**:26–49. https://doi.org/10.2478/popets-2019-0035.
36. Kumar N, Rathee M, Chandran N. *et al.* CrypTFlow: secure tensorflow inference. In: *2020 IEEE Symposium on Security and Privacy (SP)*. California: IEEE, 2020, 336–53.
37. Hubara I, Courbariaux M, Soudry D. *et al.* Binarized neural networks. *Adv Neural Inf Process Syst* 2016;**29**:40.
38. Deng L, Li G, Han S. *et al.* Model compression and hardware acceleration for neural networks: a comprehensive survey. *Proc IEEE* 2020;**108**:485–532. https://doi.org/10.1109/JPROC.2020.2976475.
39. Szegedy C, Liu W, Jia Y. *et al.* Going deeper with convolutions. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. California: IEEE, 2015, 1–9.
40. Chen W, Wilson J, Tyree S. *et al.* Compressing neural networks with the hashing trick. In: *International Conference on Machine Learning*. New York: PMLR, 2015, 2285–94.
41. Han S, Mao H, Dally WJ. Deep compression: compressing deep neural networks with pruning, trained quantization and Huffman coding arXiv preprint arXiv:1510.00149. New York: ICLR, 2015.
42. Zhou A, Yao A, Guo Y. *et al.* Incremental network quantization: towards lossless CNNs with low-precision weights arXiv preprint arXiv:1702.03044. New York: ICLR, 2017, 11–36.
43. Liu B, Li F, Wang X, Zhang B, Yan J. Ternary weight networks. *ICASSP 2023 - 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Rhodes Island, Greece*. California: IEEE, 2023, 1–5.
44. Rastegari M, Ordonez V, Redmon J. *et al.* XNOR-Net: ImageNet classification using binary convolutional neural networks. In: Leibe B (ed.), *Computer Vision – ECCV 2016*. Cham: Springer International Publishing, 2016, 525–42.
45. Jacob B, Kligys S, Chen B. *et al.* Quantization and training of neural networks for efficient integer-arithmetic-only inference. In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, 2704–13. IEEE, California.
46. Finkelstein A, Almog U, Grobman M. Fighting quantization bias with bias ArXiv preprint abs/1906.03193. California: IEEE, 2019, 35–46. https://arxiv.org/abs/1906.03193.
47. Huang Z, Lu W-J, Hong C. *et al.* Cheetah: lean and fast secure {two-party} deep neural network inference. In: *31st USENIX Security Symposium (USENIX Security 22)*. California: USENIX, 2022, 809–26.
48. Huang K, Gungor M, Fang X. *et al.* Garbled circuits in the cloud using FPGA enabled nodes. In: *2019 IEEE High Performance Extreme Computing Conference (HPEC)*. Waltham, MA, USA: IEEE, 2019, 1–6.
49. Dowlin N, Gilad-Bachrach R, Laine K. *et al.* Manual for using homomorphic encryption for bioinformatics. *Proc IEEE* 2017;**105**:1–16. https://doi.org/10.1109/JPROC.2016.2622218.
50. Chandran N, Gupta D, Rastogi A. *et al.* EzPC: programmable, efficient, and scalable secure two-party computation. *IACR Cryptol ePrint Arch* 2017;**2017**:1109.

The Computer Journal, 2025, **68**, 998–1012
https://doi.org/10.1093/comjnl/bxaf019
Original Article